

- 🎯 PLAN MAESTRO DE INTEGRACIÓN — EVENTOS MICROSERVICIOS
 - 📋 RESUMEN DEL EQUIPO Y ASIGNACIONES
 - 📁 JERARQUÍA DE ARCHIVOS DEL PROYECTO
 - ⚡ ORDEN DE INTEGRACIÓN (SECUENCIA OBLIGATORIA)
 - ✅ CHECKLIST MAESTRO DE INTEGRACIÓN
 - FASE 0 — Preparación (TODO el equipo)
 - FASE 1 — Scripts (Francisco)
 - FASE 2 — Auth Service (Rachel)
 - FASE 3 — User Service + Gateway (Andrea)
 - FASE 4 — Event Service (Braylin)
 - FASE 5 — Inscripciones + Frontend (Eliana)
 - FASE 6 — Integración Final (Rachel)
 - 🛠 COMANDOS PARA RACHEL (COORDINADORA)
 - Preparar develop antes de empezar
 - Crear PR en GitHub (para cada miembro)
 - Actualizar develop local después de cada merge
 - PR final develop → main
 - Crear tag de versión
 - 🚨 RESOLUCIÓN DE CONFLICTOS
 - Si hay conflicto al mergear PR → develop
 - Regla de oro para evitar conflictos
 - 📊 ESTADO DE PUERTOS
 - 🗄 BASES DE DATOS (Railway MySQL)

🎯 PLAN MAESTRO DE INTEGRACIÓN — EVENTOS MICROSERVICIOS

Coordinadora: Rachel | Rama principal de integración: `develop` → `main`

📋 RESUMEN DEL EQUIPO Y ASIGNACIONES

#	Persona	Componente	Rama de trabajo
1	Francisco	Scripts de inicio/parada	feature/francisco-scripts
2	Rachel	auth-service/	feature/rachel-auth-service
3	Andrea	user-service/ + gateway/	feature/andrea-user-gateway
4	Braylin	event-service/	feature/braylin-event-service
5	Eliana	inscrip-service/ + frontend-v2/	feature/eliana-inscrip-frontend

JERARQUÍA DE ARCHIVOS DEL PROYECTO

```

Eventos-Microservicios/
├── auth-service/           ← Rachel
├── event-service/         ← Braylin
├── user-service/          ← Andrea
├── gateway/               ← Andrea
├── inscrip-service/       ← Eliana
├── notification-service/  ← Rachel (coordinadora)
├── frontend-v2/           ← Eliana
├── infra/                 ← Compartido (no tocar sin coordinación)
│   └── mysql/init/
├── docs/                  ← Documentación general
│   ├── integration/       ← ESTE PLAN
│   │   ├── 00_COORDINATOR.md ← Estás aquí
│   │   ├── miembros/
│   │   │   ├── 01_RACHEL.md
│   │   │   ├── 02_BRAYLIN.md
│   │   │   ├── 03_ANDREA.md
│   │   │   ├── 04_ELIANA.md
│   │   │   └── 05_FRANCISCO.md
│   │   └── arquitectura/
│   │       └── DIAGRAMAS.md
│   ├── DELIVERY_PLAN.md
│   ├── GIT_CLEANUP.md
│   └── TEAM_WORKFLOW.md
├── .github/               ← CI/CD workflows
└── Start-All.ps1          ← Francisco (raíz)

```

└─  Stop-All.ps1	← Francisco (raíz)
└─  Start-Local.ps1	← Francisco (raíz)
└─  Monitor-Services.ps1	← Francisco (raíz)
└─  Iniciar-Proyecto.bat	← Francisco (raíz)
└─  Detener-Proyecto.bat	← Francisco (raíz)
└─  docker-compose.yml	← Compartido (no tocar)
└─  init-db.sql	← Compartido (no tocar)
└─  insert_premium_events.sql	← Compartido (no tocar)
└─  .env.example	← Compartido (no tocar)
└─  .gitignore	← Compartido (no tocar)
└─  README.md	← Rachel actualiza al final

ORDEN DE INTEGRACIÓN (SECUENCIA OBLIGATORIA)

FASE 0 – Preparación (TODOS antes de empezar)

└─ Clonar repo + configurar rama personal

FASE 1 – Francisco sube scripts primero (sin dependencias)

└─ PR: feature/francisco-scripts → develop

FASE 2 – Rachel sube auth-service (base de autenticación)

└─ PR: feature/rachel-auth-service → develop

FASE 3 – Andrea sube user-service + gateway (enrutador central)

└─ PR: feature/andrea-user-gateway → develop

FASE 4 – Braylin sube event-service (depende de auth)

└─ PR: feature/braylin-event-service → develop

FASE 5 – Eliana sube inscrip-service + frontend (depende de todo)

└─ PR: feature/eliana-inscrip-frontend → develop

FASE 6 – Rachel valida develop completo y hace PR → main

CHECKLIST MAESTRO DE INTEGRACIÓN

FASE 0 — Preparación (TODO el equipo)

- ☐ Francisco confirmó su entorno Git configurado
- ☐ Rachel confirmó su entorno Git configurado
- ☐ Andrea confirmó su entorno Git configurado
- ☐ Braylin confirmó su entorno Git configurado
- ☐ Eliana confirmó su entorno Git configurado
- ☐ Repositorio compartido con todos los miembros en GitHub
- ☐ Todos clonaron el repo exitosamente
- ☐ Branch `develop` existe y está actualizada

FASE 1 — Scripts (Francisco)

- ☐ Francisco creó rama `feature/francisco-scripts`
- ☐ Francisco subió `Start-All.ps1`
- ☐ Francisco subió `Stop-All.ps1`
- ☐ Francisco subió `Monitor-Services.ps1`
- ☐ Francisco subió `Start-Local.ps1`
- ☐ Francisco subió `Iniciar-Proyecto.bat`
- ☐ Francisco subió `Detener-Proyecto.bat`
- ☐ PR creado: `feature/francisco-scripts` → `develop`
- ☐ Rachel aprobó y mergeó el PR
- ☐ `develop` actualizada con scripts

FASE 2 — Auth Service (Rachel)

- ☐ Rachel creó rama `feature/rachel-auth-service`
- ☐ Rachel subió toda la carpeta `auth-service/`
- ☐ Rachel subió toda la carpeta `notification-service/` (coordinadora)
- ☐ PR creado: `feature/rachel-auth-service` → `develop`
- ☐ PR revisado y mergeado
- ☐ `develop` actualizada con auth-service

FASE 3 — User Service + Gateway (Andrea)

- ☐ Andrea creó rama `feature/andrea-user-gateway`
- ☐ Andrea subió toda la carpeta `user-service/`
- ☐ Andrea subió toda la carpeta `gateway/`

- ☐ PR creado: `feature/andrea-user-gateway` → `develop`
- ☐ PR revisado por Rachel y mergeado
- ☐ `develop` actualizada con user-service + gateway

FASE 4 — Event Service (Braylin)

- ☐ Braylin creó rama `feature/braylin-event-service`
- ☐ Braylin subió toda la carpeta `event-service/`
- ☐ PR creado: `feature/braylin-event-service` → `develop`
- ☐ PR revisado por Rachel y mergeado
- ☐ `develop` actualizada con event-service

FASE 5 — Inscripciones + Frontend (Eliana)

- ☐ Eliana creó rama `feature/eliana-inscrip-frontend`
- ☐ Eliana subió toda la carpeta `inscrip-service/`
- ☐ Eliana subió toda la carpeta `frontend-v2/`
- ☐ PR creado: `feature/eliana-inscrip-frontend` → `develop`
- ☐ PR revisado por Rachel y mergeado
- ☐ `develop` actualizada con inscrip-service + frontend

FASE 6 — Integración Final (Rachel)

- ☐ Rachel verifica que `develop` tiene todos los componentes
- ☐ Rachel ejecuta prueba local completa con `Iniciar-Proyecto.bat`
- ☐ Todos los servicios responden en sus puertos
- ☐ Frontend carga correctamente en `localhost:5173`
- ☐ PR final: `develop` → `main`
- ☐ PR aprobado y mergeado
- ☐ Tag de versión creado en `main`

COMANDOS PARA RACHEL (COORDINADORA)

Preparar **develop** antes de empezar

```
git checkout develop
git pull origin develop
```

Crear PR en GitHub (para cada miembro)

```
# Verificar que la rama del miembro existe
git fetch --all
git branch -r | grep feature/

# Aprobar y mergear (en GitHub UI) o por CLI:
gh pr merge [numero-pr] --merge --delete-branch
```

Actualizar **develop** local después de cada merge

```
git checkout develop
git pull origin develop
```

PR final develop → main

```
# En GitHub: New Pull Request
# Base: main ← Compare: develop
# Título: "Release: Integración completa Eventos-Microservicios v1.0"

# O por CLI:
gh pr create \
  --base main \
  --head develop \
  --title "Release v1.0: Integración completa de microservicios" \
  --body "Incluye: auth-service, user-service, gateway, event-service, inscrip-
service, frontend-v2, notification-service y scripts de inicio"
```

Crear tag de versión

```
git checkout main
git pull origin main
git tag -a v1.0.0 -m "Release v1.0.0 - Integración completa"
git push origin v1.0.0
```



RESOLUCIÓN DE CONFLICTOS

Si hay conflicto al mergear PR → develop

Rachel debe:

```
# 1. Actualizar develop local
git checkout develop
git pull origin develop

# 2. Ir a la rama con conflicto (ej: feature/braylin-event-service)
git checkout feature/braylin-event-service
git pull origin feature/braylin-event-service

# 3. Hacer merge de develop en la rama del miembro
git merge develop

# 4. Git mostrará los archivos en conflicto
git status

# 5. Abrir cada archivo con conflicto y resolverlo manualmente
# Los marcadores de conflicto son:
# <<<<<<< HEAD (tu versión)
# =====
# >>>>>>> develop (versión de develop)

# 6. Después de resolver, confirmar
git add [archivos-resueltos]
git commit -m "resolve: conflictos con develop en [componente]"
git push origin feature/braylin-event-service

# 7. El PR ahora debería estar limpio para mergear
```

Regla de oro para evitar conflictos

Cada persona solo hace `git add` de SU carpeta específica. Nunca hacer `git add .` o `git add -A`



ESTADO DE PUERTOS

Servicio	Puerto	Responsable
API Gateway	8080	Andrea
auth-service	8081	Rachel
user-service	8082	Andrea
event-service	8083	Braylin
inscrip-service	8084	Eliana
notification-service	8085	Rachel
frontend-v2	5173	Eliana



BASES DE DATOS (Railway MySQL)

Base de datos	Servicio	Responsable
auth_db	auth-service	Rachel
user_db	user-service	Andrea
event_db	event-service	Braylin
inscrip_db	inscrip-service	Eliana
notification_db	notification-service	Rachel

Host Railway: shinkansen.proxy.rlwy.net:37791

Última actualización: 2026-03-13 | Coordinadora: Rachel